

Adaptive Multimode Process Monitoring Based on Mode-Matching and Similarity-Preserving Dictionary Learning

Keke Huang¹, Zui Tao, Yishun Liu, Bei Sun, Chunhua Yang², *Fellow, IEEE*, Weihua Gui, and Shiyun Hu³, *Senior Member, IEEE*

Abstract—In real industrial processes, factors, such as the change in manufacturing strategy and production technology lead to the creation of multimode industrial processes and the continuous emergence of new modes. Although the industrial SCADA system has accumulated a large amount of historical data, which can be used for modeling and monitoring multimode processes to a certain extent, it is difficult for the model learned from historical data to adapt to emerging modes, resulting in the model mismatch. On the other hand, updating the model with data from new modes allows the model to continuously match the new modes, but it may cause the model to lose the ability to represent the historical modes, resulting in “catastrophic forgetting.” To address these problems, this article proposed a jointly mode-matching and similarity-preserving dictionary learning (JMSDL) method, which updated the model by learning the data of new modes, so that the model can adaptively match the newly emerged modes. At the same time, a similarity metric was put forward to guarantee the representation ability of the proposed method for historical data. A numerical simulation experiment, the CSTDH process experiment, and an industrial roasting process experiment indicated that the proposed JMSDL method can match new modes while maintaining its performance on the historical modes accurately. In addition, the proposed method significantly outperforms the state-of-the-art methods in terms of fault detection and false alarm rate.

Index Terms—Adaptive model updating, dictionary learning, model mismatch, multimode process modeling, process monitoring.

Manuscript received 26 February 2022; revised 10 May 2022; accepted 26 May 2022. Date of publication 10 June 2022; date of current version 17 May 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62073340 and Grant 61860206014; in part by the Major Key Project of Peng Cheng Laboratory (PCL) under Grant PCL2021A09; in part by the National Key Research and Development Program of China under Grant 2019YFB1705300; in part by the Innovation-Driven Plan in Central South University, China, under Grant 2019CX020; and in part by the 111 Project, China, under Grant B17048. This article was recommended by Associate Editor T. Huang. (*Corresponding author: Yishun Liu.*)

Keke Huang is with the School of Automation, Central South University, Changsha 410083, China, and also with Peng Cheng Laboratory, Shenzhen 518055, China (e-mail: huangkeke@csu.edu.cn).

Zui Tao, Yishun Liu, Bei Sun, Chunhua Yang, and Weihua Gui are with the School of Automation, Central South University, Changsha 410083, China (e-mail: taozui21@csu.edu.cn; liuyishun@csu.edu.cn; sunbei@csu.edu.cn; ychh@csu.edu.cn; gwh@csu.edu.cn).

Shiyun Hu is with the School of Electronics and Computer Science, University of Southampton, Southampton SO17 1BJ, U.K. (e-mail: s.hu@soton.ac.uk).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TCYB.2022.3178878>.

Digital Object Identifier 10.1109/TCYB.2022.3178878

I. INTRODUCTION

NOWADAYS, industrial processes are becoming more and more complex, which leads to the fact that a small fault in large-scale industrial production may cause huge damage to the entire system. In addition, most of the actual industrial processes are carried out under extreme conditions, such as high temperature or high pressure. Some faults may lead to industrial accidents such as toxic gas leaks, fires, and explosions, resulting in economic losses and human casualties. Process monitoring can obtain the current state of the industrial process, and when a fault is monitored, the corresponding alarm will be triggered to remind the operator to deal with the fault in time [1]–[4]. Therefore, it is particularly significant to introduce effective process monitoring methods for industrial processes.

Process monitoring has become a major research topic, and a variety of methods are proposed. Process monitoring methods can be classified into three categories: 1) model-based methods; 2) knowledge-based methods; and 3) data-driven methods. Model-based methods build models based on industrial production mechanisms [5], which are suitable for small systems with sufficient mechanistic knowledge. Knowledge-based methods build models based on expert knowledge or experience. Usually, it is time consuming, and there are scenarios in which empirical knowledge is not accurate and reliable. Data-driven methods do not require precise first-principle models and extensive process knowledge, relying only on collected data to build mathematical models, which are simple and generic. In addition, with the development of the computer and sensor technology, a large amount of process data is collected, providing enough information for data-driven process monitoring. Therefore, data-driven methods are widely used in modern complex industrial processes [6], [7]. There have been some important advances in the field of data-driven process monitoring. Li *et al.* [8] proposed a process monitoring method based on principal component analysis (PCA), which promoted the development of multivariate statistical process monitoring (MSPM). Some typical MSPM methods have received a lot of attention [9]–[11]. Chen and Ge [12] innovatively developed support vector machine (SVM)-tree and SVM-forest algorithms for imbalanced process monitoring. Jiang *et al.* [13] allowed the Bayesian analysis to be applied to industrial data monitoring. These works reflect

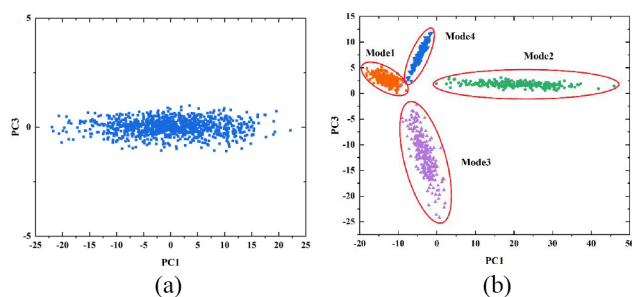


Fig. 1. Unimode and multimode data visualization. (a) Principal components of unimode data. (b) Principal components of multimode data. For multimode data, different modes show different distributions.

the malleability of machine learning in the field of process monitoring.

Dictionary learning is a powerful machine-learning method, the goal of which is to train a dictionary with the ability to represent data. Each column of the dictionary matrix is called an atom, and a small number of atoms are used to represent data linearly. Dictionary learning continues to develop for improving its effectiveness, and the K-SVD method [14], label consistent K-SVD method [15], supervised dictionary learning method [16], and discriminative K-SVD method [17] have been proposed. Some works consider the physical meaning of atoms, such as low rank, nonstationary, etc., which impose these physical constraints on dictionary atoms and achieve good results [18], [19]. But atoms are treated as abstract features in this work. Dictionary learning has a wide range of applications in the field of process monitoring. Peng *et al.* [20] proposed a method based on sparse modeling and dictionary learning for monitoring non-Gaussian processes with multiple operating conditions. Huang *et al.* [21] proposed a distributed dictionary learning method for effectively monitoring modern industrial processes with complex, distributed, and high-dimensional characteristics. Huang *et al.* [22] proposed a kernel dictionary learning method for nonlinear process monitoring.

In the process industry, a mode is defined as the process at a certain operating condition. The modes of actual industrial processes change due to, for example, market demand, product specifications, raw materials, manufacturing strategies, and fluctuations in the external environment, which leads to the continuous creation of new modes [23], [24]. Taking the roasting process of a zinc smelting factory in Hunan Province, China as an example, the roasting process can be divided into the efficient condition and healthy condition due to the process indicators and manufacturing strategies in different periods. If the conditions of the roasting process are classified according to the chemical reactions, it can be divided into over-decomposition condition, under-oxidation condition, and bed deposition condition. Therefore, the roasting process data is of complex mode. Multimode data are difficult to model and monitor because their different modes have different distribution characteristics, as shown in Fig. 1. To address this problem, there are currently three types of methods, including the multiple model method, global modeling method, and online model updating method.

The philosophy of the multiple model method is straightforward by using multiple models to match each mode, which has been widely used due to its simple idea and good performance. Zhao *et al.* [25] proposed a multiple PCA method to monitor industrial processes with multiple operating modes. Natarajan and Srinivasan [26] proposed a multimodel method to monitor the state of oil and gas production processes. Zhao *et al.* [27] proposed a multistage statistical analysis method for monitoring the penicillin culture process. However, the multiple model method also has obvious disadvantages. If new modes are continuously generated, the number of models will continuously grow and with high space complexity. In addition, such methods often require a significant amount of time to find the most appropriate model to match the data when online monitoring data arrives.

The global modeling method devotes to build a single model to represent the data of all modes simultaneously. Hwang and Han [28] proposed a process monitoring method with multiple modes based on hierarchical clustering and PCA. Xu *et al.* [29] proposed a PCA mixture model for the early detection of various faults in multimode processes. Ning *et al.* [30] proposed a label-consistent dictionary learning (LCDL) method for monitoring the chemical industry. The global modeling method solves the problem of an excessive number of models in the multiple model method. However, when the number of modes is large, the number of training samples can also be very large, leading to time consuming in training. Moreover, due to the statistical averaging effect, the global model may not accurately represent each mode. In addition, when a new mode appears, the previously trained model can no longer be used, and the new mode data should be added to the training set to retrain the model, which is a very tedious and time-consuming process.

The online model updating method enables the model to constantly adapted to new modes. This type of method usually trains an initial model using historical data and updates the model online using data from the newly emerged mode, so that the model has a good representation of the new mode data. Jeng [31] proposed a recursive PCA and moving window PCA technique to update the model and the control limit online. Huang *et al.* [32] developed an online dictionary learning (ODL)-based process monitoring technique. This type of method enables model updates and solves the problem that the model needs to be retrained when a new mode appears. However, online model updating methods are often not adaptable to processes with large mode variations. Moreover, some online model updating methods may face the problem of “catastrophic forgetting,” that is, as the model continuously updates, it may lose its ability to represent the previous modes’ data. Fig. 2 shows the reconstruction results of the ODL method for the multimode data in Fig. 1. As the model is updated, its reconstruction error of the previous data increases, which is a typical phenomenon of “catastrophic forgetting.” The reason for “catastrophic forgetting” is that the model may overwrite previously learned knowledge when it is updated or retrained. It introduces difficulties for process monitoring. If the updated model experiences “catastrophic forgetting,” the previous modes can no longer be monitored.

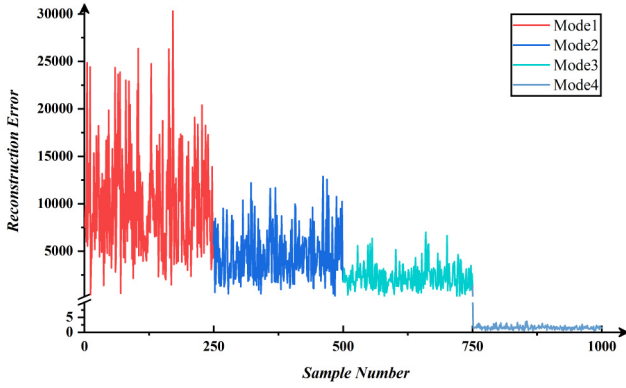


Fig. 2. “Catastrophic forgetting” phenomenon of the ODL method [32]. As data of new modes are continuously learned, the model forgets how to represent the previous data, resulting in high reconstruction errors.

To solve the multimode process monitoring with continuously increasing modes and eliminate the negative effect of “catastrophic forgetting” simultaneously, a jointly mode-matching and similarity-preserving dictionary learning (JMSDL) method is proposed in this article. The overcomplete dictionary is commonly used in dictionary learning, which results in enough redundant space in the dictionary to store the features of multiple modes. Therefore, dictionary learning has a clear advantage in dealing with multimode data, using only a dictionary that can represent data from multiple modes. Our method first trains an initial model using the K-SVD method [14] on the data of the first mode. When a new mode appears, the dictionary is updated using only the data of the new mode through the JMSDL method so that the dictionary can represent the data of the new mode without losing its ability to represent the data of the historical modes. By updating the dictionary using the data of all modes sequentially, we finally obtain a dictionary that can represent all modes and use it to monitor the multimode process. The main contribution points of this article are as follows.

- 1) This article proposes the JMSDL method to adaptively update the process monitoring model by minimizing the reconstruction error of the new mode data to ensure model matching.
- 2) The proposed JMSDL method overcomes “catastrophic forgetting” by putting forward a preservation term in the optimization function to ensure the similarity of the model before and after the updating.
- 3) The proposed JMSDL method provides a continuous learning framework for multimode process monitoring that is adaptive to new modes.

The remainder of this article is structured as follows. The specific method introduction is presented in detail in Section II. In Section III, examples of a numerical simulation and the CSTD process are used to verify the superiority of the proposed method in data representation and process monitoring. Section IV introduces the application of the JMSDL-based process monitoring method in the real roasting process. Finally, the conclusion is given in Section V.

II. METHODOLOGY

A. Offline Modeling

The entire process of offline modeling can be divided into two parts: 1) initial modeling and 2) adaptive model updating. First, multimode data are collected. Mathematically, the dataset is denoted as $X = [X_1, X_2, \dots, X_s]$, with a total of s modes. For each of these modes, $X_i = [x_1, x_2, \dots, x_M] \in R^{m \times M}$, $i = 1, \dots, s$. m and M denote the dimensionality and number of samples, respectively. In the initial modeling stage, an initial dictionary is trained using the data of the first mode X_1 based on the traditional dictionary learning method. In the adaptive model updating stage, the dictionary is updated based on the proposed JMSDL method, using only the data from the new mode. The updated dictionary can represent the data of the new mode and historical modes. In this section, the process of modeling multimode industrial process data in the offline stage is described in detail in two parts.

1) *Initial Modeling*: The task of dictionary learning is to learn a dictionary $D = [d_1, d_2, \dots, d_K] \in R^{m \times K}$, with K denotes the number of atoms. To facilitate searching for the optimal atom, unitization is performed for each atom with $\|d_i\|_2^2 = 1 \forall i$. Dictionary learning uses a small number of atoms to linearly reconstruct the data, with $x_i \approx Dw_i$. w_i is a sparse coding vector. Extend to the entire dataset, that is, $X_1 \approx DW$. $W = [w_1, w_2, \dots, w_M] \in R^{K \times M}$ is called the sparse coding matrix. The optimization problem of dictionary learning is as follows:

$$\langle D, W \rangle = \arg \min_{D, W} \|X_1 - DW\|_F^2; \text{ s.t. } \forall i, \|w_i\|_0 \leq T \quad (1)$$

where $\|X_1 - DW\|_F^2$ denotes reconstruction error. $\|\cdot\|_F$ denotes the Frobenius norm of the matrix. If $A = [a_{ij}]_{m \times n}$, $\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$. Minimizing the reconstruction error ensures that the dictionary has adequate representation ability to the data. $\|w_i\|_0$ is the L_0 norm of the vector w_i , which counts the number of nonzero values in the vector. T denotes the sparsity. The L_0 norm constraint to the coding vector ensures that no more than T atoms are used to reconstruct the data. Mathematically, it is common to approximately replace L_0 norm with L_1 norm, and to transform the constrained optimization problem of (1) into an unconstrained optimization problem as follows:

$$\langle D, W \rangle = \arg \min_{D, W} \left\{ \|X_1 - DW\|_F^2 + \lambda \|W\|_1 \right\}. \quad (2)$$

There are two optimization variables D and W in (2), and they are not joint convex, which can be solved by the alternating iteration method, where each iteration step solves for the current optimal D and W , respectively. Fix W when optimizing D , and use the K-SVD method [14] to solve for the current optimal D ; fix D when optimizing W , and use the orthogonal matching pursuit (OMP) method [33] to solve for the current optimal W . When a sufficient number of iterations is reached or the convergence condition is met, a dictionary with sufficient data representation capability is obtained.

The initial dictionary D has a good representation ability to the data of mode 1. In fact, we collected data from multiple modes but used only the data from mode 1 when training the

initial dictionary. We do not use data from multiple modes to train the initial dictionary, because the K-SVD method is a method for unimode data. If we force to use multimode data, it is likely that the dictionary will not represent well for all modes due to statistical averaging. In order for the dictionary to eventually have the ability to represent multimode data, the dictionary will be updated adaptively using data from other modes after the initial dictionary is obtained.

2) *Adaptive Model Updating*: In real industrial processes, a model often cannot match every stage of the process due to the changing working conditions, so the model needs updating. However, it is often not possible to use all the historical data in the model update. This is because as the industrial process continues to run, more and more data will be available, causing the update to take up a large number of resources. In addition, in industrial sites, outdated historical data may be deleted. Therefore, in this article, we investigate incrementally updating the model without using old data and addressing the possible “catastrophic forgetting” during the update.

Define the dictionary before the update as the old dictionary D_o , and the dictionary after the update as the new dictionary D_n . D_o has been trained using the dataset X_o that belongs to historical modes. X_n denotes the data of the new mode. Referring to the philosophy of incremental learning [34], the proposed JMSDL method is to use X_n to update D_o to obtain a new dictionary D_n that can adapt to the new mode data X_n without losing the ability to represent the old data X_o . The JMSDL method enables the representation of multimode data and the dictionary can be continuously updated. Therefore, the goal of JMSDL can be divided into two parts. The first part called mode-matching is that the dictionary can adapt to new mode data. The second part is to maintain the dictionary’s ability to represent old data and eliminate “catastrophic forgetting,” which is called similarity preserving. The JMSDL is modeled by designing the constraint terms corresponding to the two parts, as shown in Fig. 3.

To achieve mode-matching, the dictionary needs to represent the new data well, that is $X_n \approx D_n W$. In the same way as traditional dictionary learning, the reconstruction term $\|X_n - D_n W\|_F^2$ is defined, and the representation capability of D_n to X_n is ensured by minimizing the reconstruction error. In addition, the dictionary needs to maintain the ability to represent the old data but no longer use the old data when updating the dictionary. Since D_o can represent old data, we keep D_n similar enough to D_o when training D_n to ensure the representation capability of D_n to old data and achieve similarity preserving. So we define the similarity matrix

$$D_o^T D_n = \begin{bmatrix} d_{o1}^T d_{n1} & d_{o1}^T d_{n2} & \cdots & d_{o1}^T d_{nK} \\ d_{o2}^T d_{n1} & d_{o2}^T d_{n2} & \vdots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ d_{oK}^T d_{n1} & \cdots & \cdots & d_{oK}^T d_{nK} \end{bmatrix} \quad (3)$$

where the elements on the diagonal are the inner products of the same positioned atoms of both dictionaries. The atoms are all unit vectors, so when two unit vectors are very similar, their inner product will approach 1. We want there is sufficient similarity between the atoms at the same position in

both dictionaries. The similarity of the dictionaries is reflected quantitatively on the similarity matrix, where each diagonal element of the similarity matrix is close to 1. Since the diagonal elements of the unit matrix are 1, disregarding the elements at other positions of the similarity matrix, we expect its diagonal to be similar to the unit matrix. Let I is the unit matrix and construct the preservation term $\text{tr}(I - D_o^T D_n)$. We call this operation the diagonal unitization of the similarity matrix, which forces the diagonal elements of the similarity matrix to approach 1 to maintain the similarity between D_o and D_n . In summary, the optimization problem of JMSDL is obtained as follows:

$$\langle D_n, W \rangle = \arg \min_{D_n, W} \left\{ \|X_n - D_n W\|_F^2 + \lambda_1 \text{tr}(I - D_o^T D_n) + \lambda_2 \|W\|_1 \right\}. \quad (4)$$

Next, we designed an optimization algorithm to solve (4). According to the philosophy of alternating optimization, variables can be optimized one by one by fixing other variables when optimizing one variable [35]. The optimization process of (4) is divided into two parts: 1) updating D_n and 2) updating W . The detailed optimization process is given as follows.

1) *Updating D_n* : By fixing W , and keeping the part of (4) related to D_n , the following suboptimization problem can be obtained:

$$D_n = \arg \min_{D_n} \left\{ \|X_n - D_n W\|_F^2 + \lambda_1 \text{tr}(I - D_o^T D_n) \right\}. \quad (5)$$

Since the Frobenius norm is convex with trace, the partial derivative of (5) to D_n is obtained as follows:

$$D_n W W^T = X_n W^T + \frac{1}{2} \lambda_1 D_o. \quad (6)$$

Let $B = W W^T$ and $F = X_n W^T + (1/2) \lambda_1 D_o$, rewrite (6) as follows:

$$D_n B = F. \quad (7)$$

Since B is a real symmetric matrix, the orthogonal diagonal decomposition of B is given

$$B = M V M^T \quad (8)$$

where V is a diagonal matrix and $M M^T = I$. Substituting (8) into (7) as follows:

$$D_n M V M^T = F. \quad (9)$$

Rewrite (9) as follows:

$$D_n M V = F M. \quad (10)$$

Then, let $D_n M = Q$ and $F M = P$, (10) can be rewritten as follows:

$$Q V = P. \quad (11)$$

Let $[\mu_1, \mu_2, \dots, \mu_K]$ be the vector consisting of the diagonal elements of V , and q_{ij} and p_{ij} be the elements in Q and P , respectively, according to (11), we get the formula as follows:

$$q_{ij} \mu_j = p_{ij} \quad \forall i, j. \quad (12)$$

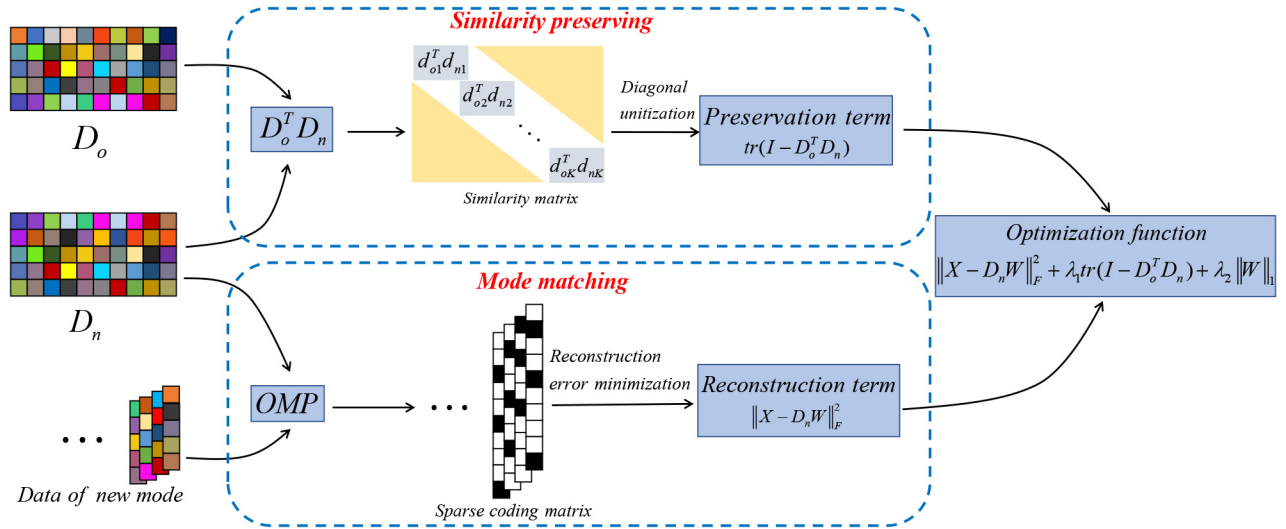


Fig. 3. Framework of mode-matching and similarity-preserving dictionary learning. The framework includes a reconstruction term, which is used to learn new knowledge, and a preservation term, which is used to eliminate the forgetting to the old knowledge by preserving the similarity of the model before and after updating.

Further, the solution equation for q_{ij} can be obtained as follows:

$$q_{ij} = \frac{p_{ij}}{\mu_j}. \quad (13)$$

Using (13) to solve for each element of Q , the update formula for the dictionary D_n is then obtained as follows:

$$D_n = QM^T. \quad (14)$$

2) *Updating W* : Fixing D_n , and keeping the part of (4) related to W , we obtain the suboptimization problem as follows:

$$W = \arg \min_W \left\{ \|X_n - D_n W\|_F^2 + \lambda_2 \|W\|_1 \right\}. \quad (15)$$

Using the OMP method, the optimization problem of (15) can be solved directly to obtain the sparse coding matrix W .

Steps 1) and 2) are iterated sequentially until the number of iterations is reached or the convergence condition is satisfied. Although (4) is not joint convex for (D_n, W) , it is convex with respect to each of them when the others are fixed. In this way, the convergence of the designed optimization method can be guaranteed, and the convergence was studied in [14] and [36]. The detailed steps of the optimization algorithm of JMSDL are summarized in Algorithm 1.

The dictionary is updated using the JMSDL method. During each updating process, the dictionary adaptively learns a new mode. A total of s historical modes are collected. For the first mode, an initial dictionary is learned using the K-SVD method; and for the remaining $s - 1$ modes, the dictionary is updated sequentially $s - 1$ times using the proposed JMSDL method, and the steps of dictionary updating are shown in Fig. 4. Compared with the traditional multimode methods, the proposed method achieves adaptive updating of the model to adapt to the new modes that constantly appear in the actual industrial process. In addition, the incremental update mechanism greatly saves training resources.

Algorithm 1 JMSDL Method

Input: Date of new mode X_n , old dictionary D_o , sparsity T .
Initialization: $D_n = D_o$, using the OMP method to obtain the initial W , $k = 0$.
While $k < \text{iteration}$ **do**
 Step 1: Fix W , $B = WW^T$ and $F = X_n W^T + \frac{1}{2} \lambda_1 D_o$;
 Step 2: Decompose B according to (8) to calculate M and V ;
 Step 3: $Q = D_n M$ and $P = FM$;
 Step 4:
 For $i = 1$ **to** m **do**
 For $j = 1$ **to** K **do**
 $Q(i, j) = P(i, j) / V(j, j)$;
 End For
 End For
 Step 5: $D_n = QM^T$;
 Step 6: Normalize atoms of D_n . $d_i = \frac{d_i}{\|d_i\|_2} \forall i$;
 Step 7: Using the OMP method to obtain W ;
 Step 8: $k = k + 1$;
end while
Output: Updated dictionary D_n

B. Online Monitoring

In the online monitoring stage, we use a trained dictionary to monitor the state of the industrial process. Other variables in the model, such as W , do not need to be stored. Specifically, when online data arrives, we use the dictionary to reconstruct the data and determine whether the data is normal or faulty based on the magnitude of the reconstruction error. In this section, the calculation of the control limit is described in detail, as well as how to reconstruct online data using the dictionary and how to determine the state of the data.

1) *Calculating Control Limit*: After learning all the historical modes data, the current dictionary D_c is obtained. Use D_c to reconstruct all training data that have been learned and

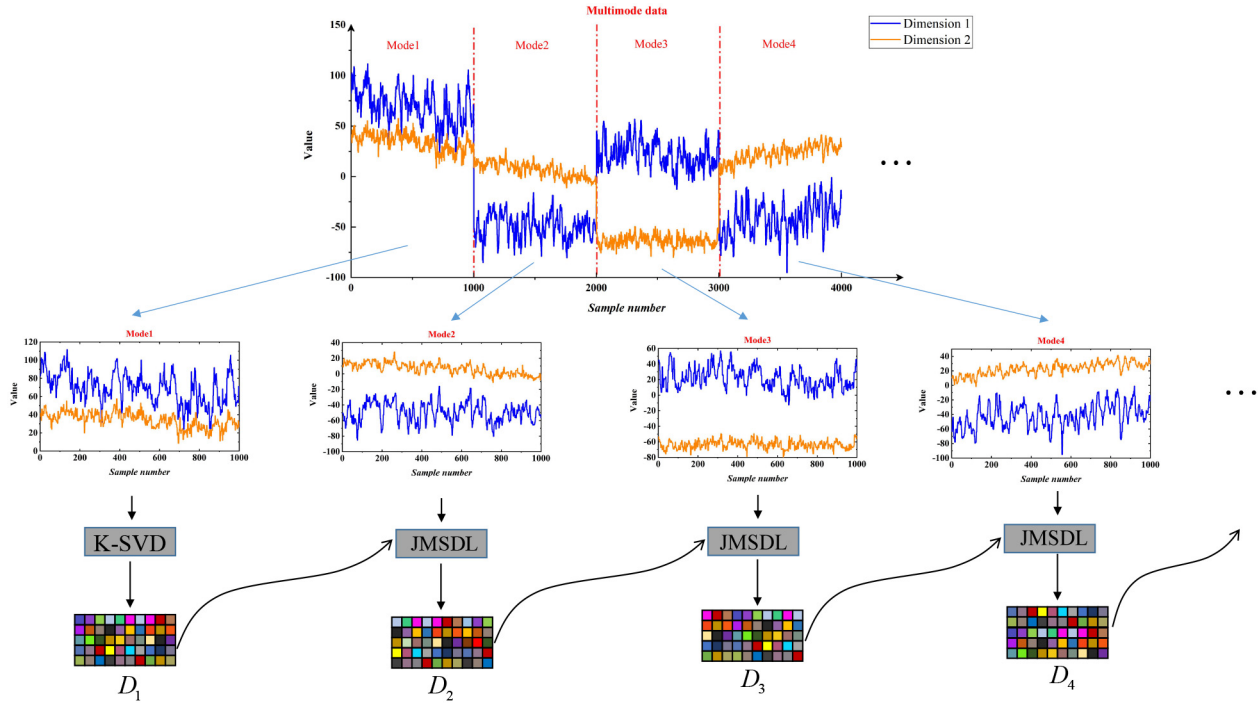


Fig. 4. Steps of updating the dictionary. First, D_1 is obtained based on the K-SVD method using the data of mode 1; then obtain D_2 based on the proposed JMSDL method using the data of mode 2 and D_1 ; then obtain D_3 using the data of mode 3 and D_2 ; then obtain D_4 using the data of mode 4 and D_3 . This process of dictionary updating can continue once data of a new mode is collected.

calculate the reconstruction error for each training sample

$$R_i = \|x_i - D_c w_i\|_2^2 \quad (16)$$

x_i denotes the i th training sample and R_i denotes the reconstruction error of the i th sample. w_i denotes the sparse coding coefficient of the i th sample, which can be calculated using the OMP method. After the reconstruction errors of all training samples are calculated, they are used to calculate the control limit R_{tr} . In this article, based on the reconstruction errors of the training data, we use the kernel density estimation (KDE) method [37] to calculate R_{tr} . The expression of the KDE method is as follows:

$$f(R) = \frac{1}{Mh} \sum_{i=1}^M K\left(\frac{|R - R_i|}{h}\right) \quad (17)$$

where M denotes the number of training samples and h denotes the bandwidth that can be automatically selected. $f(R)$ is the density function and $K(x)$ denotes the Gaussian kernel function. The control limit R_{tr} can be easily obtained using the KDE method by choosing an appropriate confidence level α . The calculation formula is as follows:

$$\int_{-\infty}^{R_{tr}} f(R) dR = \frac{\alpha}{2}. \quad (18)$$

Since the dictionary trained by the JMSDL method has the same representation capability for normal data of all modes, the magnitude of the reconstruction errors of different modes is kept at the same level. Here, we calculate a global control limit using the reconstruction errors of all modes data. When a new mode arrives and D_c is updated, we recalculate the reconstruction errors and update the control limit again by the

KDE method. The updating of D_c and the control limit is synchronized.

2) *Monitoring Online Data:* When online monitoring data x_{new} arrives, we do not need to know the mode label of the data and directly reconstruct the data using the dictionary D_c since it has equal representation capability for all learned modes. Based on the sparse representation model, the sparse coding is calculated as follows:

$$w_{new} = \arg \min_w \|x_{new} - D_c w\|_2^2 \text{ s.t. } \|w\|_0 \leq T. \quad (19)$$

The above optimization problem can be solved using the OMP method. After obtaining the sparse coding, the reconstruction results for the online data can be obtained as follows:

$$\hat{x}_{new} = D_c w_{new}. \quad (20)$$

After that, we can calculate the reconstruction error *IRE* of the online data as follows:

$$IRE_{new} = \|x_{new} - \hat{x}_{new}\|_2^2. \quad (21)$$

Process monitoring is performed by calculating the *IRE* of the online data. The dictionary has learned the features of the training data, so it knows how to represent this class of data. If the online data is of the same class as the normal data used for training, it can also be well reconstructed by the dictionary and its *IRE* will be smaller. Faulty data is different from normal data. The dictionary does not know how to represent it, so the *IRE* is large. Previously, we have calculated the control limit R_{tr} using the reconstruction errors of the training data. Therefore, the current data is determined as normal or faulty by comparing IRE_{new} with the control limit R_{tr} . If $IRE_{new} > R_{tr}$,

we consider the current online data to be faulty. Otherwise, the data is normal.

In summary, for multimode processes, we first train an initial dictionary using data from one mode using the K-SVD method. Then based on the initial dictionary, the dictionary is updated by the JMSDL method, and each updating process learns the data of a new mode. Finally, a dictionary with representation capability for all modes data is obtained. The control limit is calculated based on the reconstruction errors of the training data. In the online monitoring stage, the reconstruction error of the online data is calculated and compared with the control limit to determine whether the data is faulty or not.

III. EXPERIMENTAL RESULTS AND ANALYSIS

In this section, a numerical simulation experiment and the CSTD simulation process experiment are presented to illustrate the superiority of the proposed JMSDL method. Some indexes are first defined to quantify the superiority of the proposed method. To quantitatively evaluate the similarity between the old dictionary and the updated dictionary, dictionary similarity ds is defined

$$ds = \frac{1}{K} \sum_{i=1}^K |d_{oi}^T d_{ni}| \quad (22)$$

where K denotes the number of atoms in the dictionary, d_{oi} denotes the i th atom of the old dictionary, and d_{ni} denotes the i th atom of the updated dictionary. ds is a value in the range of 0 to 1. The closer the value is to 1, the more similar the two dictionaries are. To quantitatively evaluate the dictionary's ability to represent each mode data, the mean reconstruction error MRE is defined

$$\text{MRE} = \frac{1}{m} \sum_{i=1}^m \text{IRE}_i \quad (23)$$

where IRE_i denotes the reconstruction error of the i th sample and m denotes the number of samples. The smaller the MRE, the better the dictionary's ability to reconstruct this mode. To quantitatively evaluate the effectiveness of the various process monitoring methods, the fault detection rate and the false alarm rate are defined here

$$\text{FDR} = \frac{tp}{\text{total count of faulty data}} \times 100\% \quad (24)$$

$$\text{FAR} = \frac{fp}{\text{total count of normal data}} \times 100\% \quad (25)$$

where tp denotes the number of correctly detected faulty samples and fp denotes the number of normal samples that were misclassified as faulty data.

To demonstrate the superiority of our method, four state-of-the-art process monitoring methods are selected here for comparison, including the PCA mixture model (mPCA) method [29], the traditional dictionary learning (DL) method [14], the LCDL method [30], and the ODL method [32]. The DL method is involved in the comparison as the most typical dictionary learning method. We would like to compare the performance of the proposed method with the

global modeling method for multimode process monitoring. Therefore, we choose the LCDL method, which is a global modeling method based on dictionary learning that uses a dictionary model to represent the data of all modes. In addition, PCA is an important process monitoring method. We choose an advanced PCA method for multimode processes, namely, mPCA, to participate in the comparison. We also chose the ODL method, which is an advanced online updating method. It has many commonalities with the proposed method. Both can continuously learn new knowledge. But unlike ODL, the proposed JMSDL method considers that the model does not forget the old knowledge while learning the new knowledge. We implement this function with a preservation term, which is not available in ODL.

A. Numerical Simulations

A multimode data generation system is designed to test the performance of the proposed method on multimode process data. Since multimode data require the data of different modes to show different distributions, we generate the data as follows:

$$x = A_i s + e \quad (26)$$

where $A_i \in R^{20 \times 2}$ is a random observation matrix and different mode with different A_i . $s = [s_1, s_2]^T$ denotes the state vector of the process, in this experiment, $s_1 \sim N(2, 1)$, and $s_2 \sim N(3, 1)$. $e = [e_1, e_2, \dots, e_{20}]^T$ denotes the Gaussian noise, where $e_i \sim N(0, 0.1) \forall i$. $x = [x_1, x_2, \dots, x_{20}]^T$ denotes simulated process data.

In general, when the observation matrix changes, the distribution of the data generated by (26) also changes, and the data will be divided into different modes. Thus, we use different random matrices A_i to obtain data for different modes. Since the state vector s obeys a normal distribution, s is different each time we use (26), ensuring that we can generate a large number of different samples for each mode. Therefore, the data generated by the above numerical simulation system fully meet the requirements of this experiment.

First, we use the numerical simulation system to verify the feasibility of the optimization algorithm. Since λ_1 is an important parameter, we conduct a sensitivity analysis experiment on λ_1 . Change λ_1 , run the optimization algorithm to get the updated dictionary, and calculate the current ds corresponding to λ_1 . To eliminate the fluctuation induced by the random initialization, we do several same experiments and calculated the mean and standard deviation of ds . We get the curve as in Fig. 5. The results in Fig. 5 show that when λ_1 increases, ds tends to increase in general and its standard deviation tends to reduce. That is, the increase of λ_1 promotes the new dictionary to be more similar to the old dictionary, which is consistent with the optimization problem of (4). In addition, to show the similarity between the old dictionary and the new dictionary more visually, we visualize the matrix $(D_o - D_n)$, as in shown Fig. 6. The values of most elements in the matrix are close to 0, and only the values of some elements in a few columns are larger. Since $(D_o - D_n)$ reflects the differences between the two dictionaries, Fig. 6 shows that most of the corresponding atoms of the two dictionaries are very similar,

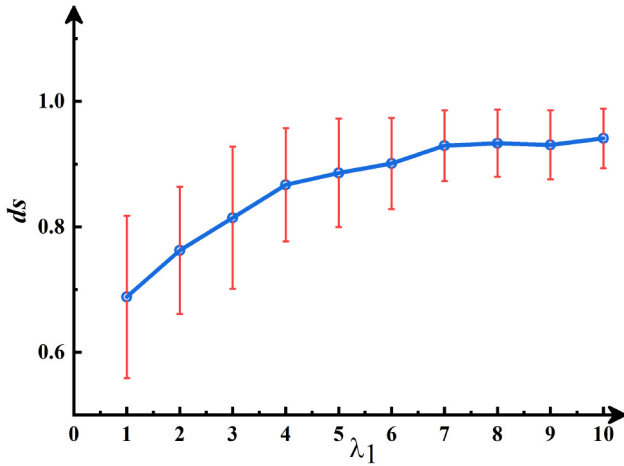


Fig. 5. Effect of different λ_1 to ds . The increase of λ_1 promotes the new dictionary to be more similar to the old dictionary.

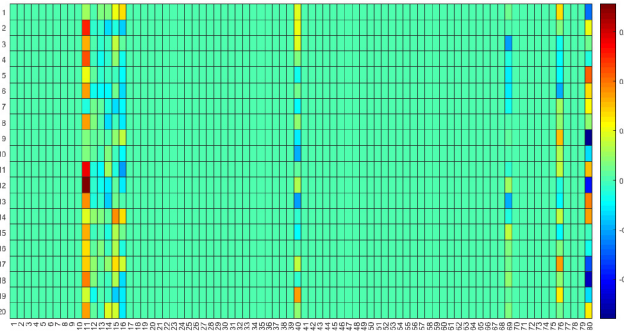


Fig. 6. Heatmap of $(D_o - D_n)$. When the dictionary is updated, a small number of atoms are rewritten, which indicates new knowledge, and most of the unchanged atoms are used to maintain the old knowledge.

and only a few are different, such as the 11th and 80th atoms. It indicates that new knowledge has been rewritten into these a few atoms after the dictionary update. The atoms that do not change much maintain the knowledge they have learned previously. In fact, it is not the case that the more similar the updated model is to the previous one, the better the monitoring effect is. Because it reflects that the model is not learning new knowledge. Therefore, λ_1 needs to be adjusted to find a suitable interval where the model learns the new knowledge and does not forget the old knowledge. The experimental results in Figs. 5 and 6 are to illustrate that the proposed optimization algorithm is feasible and can make the updated dictionary similar to the old dictionary.

After verifying the feasibility of the optimization algorithm, we use numerical simulation data to examine the ability of the JMSDL method to represent multimode data. The process of this experiment can be divided into three steps as follows.

- 1) *Data Generation*: Training data and test data are generated by (26). In this experiment, we use four different random matrices A_1 to A_4 to generate data for four modes in total. There are 1000 samples as training data and 250 samples as test data for each mode. In total, there are 4000 training samples and 1000 test samples. In the test dataset, the data are arranged in the order of mode 1, mode 2, mode 3, and mode 4.

- 2) *Model Training*: The steps of model training and updating are as Fig. 4. First, D_1 is obtained based on (2) using the data of mode 1; then obtain D_2 based on (4) using the data of mode 2 and D_1 ; then obtain D_3 using the data of mode 3 and D_2 ; then obtain D_4 using the data of mode 4 and D_3 .
- 3) *Data Representation*: After obtaining D_1 to D_4 separately, four dictionaries are used to represent the test dataset. For each dictionary, the sparse coding of the test data is obtained by (19), and then the IRE is calculated by (21). Then the mean reconstruction error MRE of the dictionary for each mode is calculated by (23), to evaluate the ability of the dictionary to represent the data of different modes.

The dictionary size and λ_1 are two important hyper-parameters. We find the optimal combination of parameters using grid search after using a small amount of data as the validation set. The sparsity should often be less than 10% of the dictionary size. The parameters of the experiment are set as follows. Each dictionary has the same size, and the number of dictionary atoms is 80. The sparsity for mode 1 is 3, that is, a linear combination of three atoms in D_1 is used to represent each data. When D_2 is obtained using the JMSDL method, λ_1 is 3 and the sparsity is 3. When training D_3 , λ_1 is 2.5 and the sparsity is 3. When training D_4 , λ_1 is 2.6 and the sparsity is 5.

To verify the role of the preservation term in the JMSDL method, a comparative experiment of data representation is conducted. When the preservation term is removed, the JMSDL method degenerates into traditional dictionary learning. Therefore, we compared the performance of JMSDL and DL methods on test data when learning the four modes continuously, as shown in Fig. 7. It can be seen that DL faces severe catastrophic forgetting, and whenever it has finished learning the data of a mode, it tends to forget how to represent the previous modes, causing an increase in reconstruction error. In contrast, the JMSDL method with the preservation term has better continuous learning capability. More intuitively, we show the MRE indicators of the four dictionaries obtained by JMSDL for the four modes, as shown in Fig. 8. D_1 can only represent mode 1. Because it only learns how to represent the data of mode 1, it has a relatively large reconstruction error for the other modes. Based on D_1 , the dictionary is updated to D_2 using the training data of mode 2. D_2 can represent modes 1 and 2. Similarly, D_3 can represent mode 1, 2, and 3; D_4 can represent all four modes. In addition, when a mode is not learned, the reconstruction error is very large. This is due to two reasons. One is that the dictionary has not learned how to represent this mode. The second is that the dimensionality of the data for the numerical simulation and the values of the data themselves are on the large side. Through the data representation experiment, we find that the JMSDL method can learn the data of a new mode each time when updating. The updated dictionary also has good representation capability for historical modes, which is the role of the preservation term. The JMSDL method does not suffer from “catastrophic forgetting.”

Next, we conduct the process monitoring experiment for the JMSDL method and the compared methods. Since D_4

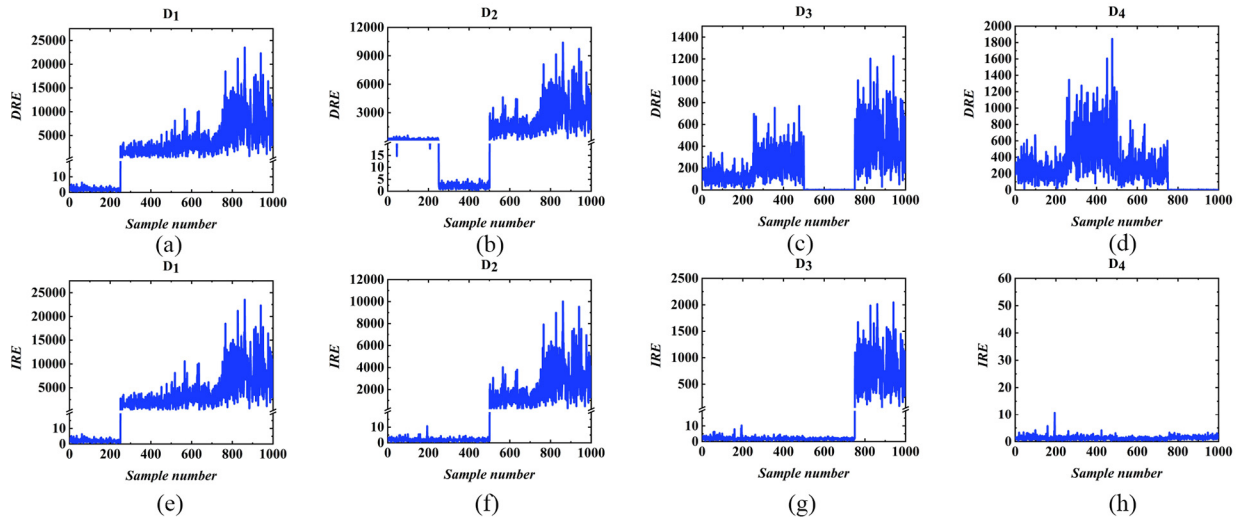


Fig. 7. Comparison of representation effect for different dictionaries on numerical simulation test data. (a)–(d) *DRE* statistic on test data for traditional DL method with continuous learning of four modes. (e)–(h) *IRE* statistic on test data for the proposed JMSDL method with continuous learning of four modes. The proposed method avoids the “catastrophic forgetting” problem faced by traditional dictionary learning.

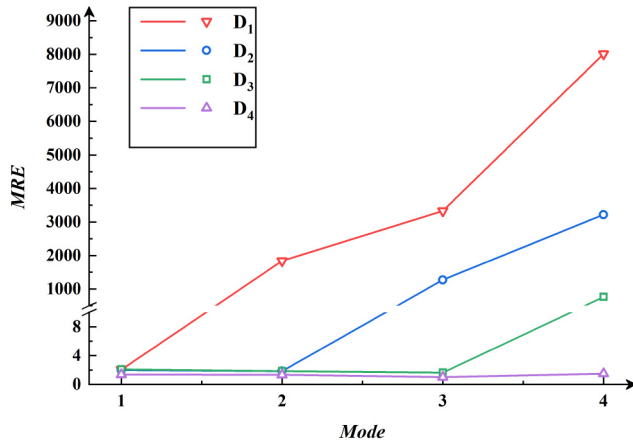


Fig. 8. Mean reconstruction errors of the four dictionaries for four modes of numerical simulation data. The smaller the MRE, the better the representation of this mode. JMSDL can continuously learn new modes and maintain a small MRE for the previous modes.

can represent the data for all modes, process monitoring of this multimode numerical simulation process is performed using D_4 . The process monitoring experiment can likewise be divided into three steps, including data generation, model training, and online monitoring. In the process monitoring experiment, the training dataset is kept constant. For the test dataset, +4 bias faults are added to x_2 for the last 125 data of each mode. Thus, there are 500 normal samples and 500 faulty samples in the test data. During the model training step, in addition to obtaining D_4 , it is necessary to calculate the control limit. The reconstruction errors of D_4 for all training samples should be calculated, and then the control limit R_{tr} is calculated using the KDE method. In the online monitoring step, the OMP method is used to obtain a sparse coding of D_4 for each test data, and then the reconstruction error *IRE*

is calculated. Compare *IRE* with R_{tr} to determine whether the online data is faulty. The mPCA, DL, LCDL, and ODL methods are used as compared methods. For the four compared methods, the models are trained using the training data of all modes, and then the process monitoring is performed on the test data. The process monitoring effect of different methods is judged by comparing the FAR and FDR. The parameters of each method are set as follows. The CPV of the mPCA method is 0.85. For all the DL-based methods, the number of atoms is 80 and the sparsity is 3. The confidence level for all methods is 0.99.

Fig. 9 shows process monitoring effects of different methods on numerical simulation data. Although mPCA is a multimode method, it also faces the problem of statistical averaging, which causes it to lose its effect on some modes. DL is an unimode method that does not work well when dealing with multimode data. LCDL is a global modeling method that uses a dictionary to represent the data of all modes. In this experiment, it is found that the LCDL method has the same level of reconstruction errors for mode 1, mode 3, and mode 4, but the reconstruction error is larger for mode 2, which leads to an excessive control limit, resulting in a very low FDR. The ODL method is found to have several shortcomings when monitoring numerical simulation data. First, the ODL method may learn faulty data as normal data when learning test data online, resulting in the updated model failing to distinguish faults. Second, although the data of all modes are learned during the initial modeling stage, as the model is updated, the model suffers from “catastrophic forgetting,” resulting in particularly large reconstruction errors when new modes appear, and only after online learning some new mode data can the reconstruction errors be reduced. The proposed method has small reconstruction errors for normal data of each mode and large reconstruction errors for faulty data. In addition, the reconstruction errors for different modes remain in the same

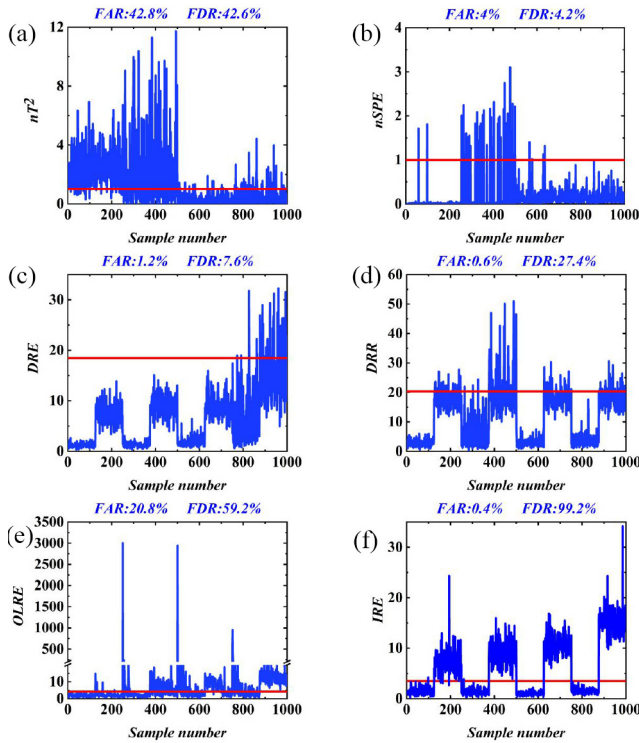


Fig. 9. Process monitoring effects of different methods on numerical simulation data. (a) nT^2 statistic of the mPCA method. (b) $nSPE$ statistic of the mPCA method. (c) DRE statistic of the DL method. (d) DRR statistic of the LCDL method. (e) $OLRE$ statistic of the ODL method. (f) IRE statistic of the proposed JMSDL method.

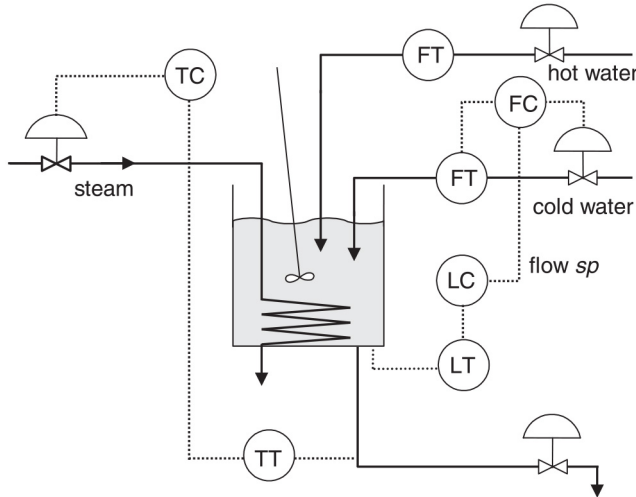


Fig. 10. Schematic of the CSTD process.

range. Therefore, the proposed method can well distinguish normal data as well as fault data and has a good process monitoring effect.

B. CSTD Simulation Process Experiment

The CSTD process is a widely used industrial process benchmark. The schematic of the CSTD process is shown in Fig. 10. In this experiment, we use the CSTD simulation

TABLE I
PARAMETER SETTING FOR THREE MODES

Mode	Level	Temperature	Hot water valve
Mode1	12.0	10.5	5.5
Mode2	16.0	10.5	5.0
Mode3	9.0	10.5	4.5

TABLE II
MRE STATISTICAL OF THREE DICTIONARIES FOR EVERY MODE OF CSTD PROCESS

Dictionary	MRE (Mode 1)	MRE (Mode 2)	MRE (Mode 3)
D_1	0.00047	3.2891	4.7353
D_2	0.00096	0.0014	15.4292
D_3	0.0013	0.0044	0.0021

TABLE III
THREE CASES OF FAULTS TO THREE MODES IN CSTD PROCESS

Mode	Variable	Fault
Mode1	Level	A bias fault is added with 1
Mode2	Flow	A multiplicative fault is imposed with 1.07
Mode3	Temperature	A multiplicative fault is imposed with 1.1

system to generate three modes. The detailed parameter setting for these modes is given in Table I.

For each mode, there are 1000 normal samples as training data and 250 normal samples as test data. First, the data of mode 1 is trained using the K-SVD method to obtain D_1 . The number of atoms is 80 and the sparsity is 4. Then, the dictionary is updated to D_2 based on the JMSDL method using the data of mode 2. The sparsity is 4 and λ_1 is 0.05. The dictionary is updated from D_2 to D_3 based on the JMSDL method using the data of mode 3. The sparsity is 4 and λ_1 is 0.28.

We use these three dictionaries to represent the test data, Fig. 11 shows the representation effect. D_1 can only represent mode 1; D_2 can represent mode 1 and mode 2; D_3 can represent all three modes. To quantitatively show the effect of the dictionary representing each mode, we calculated the MRE of three dictionaries for each mode, as shown in Table II. Updating the dictionary with each new mode reduces the reconstruction error of that mode while maintaining the dictionary's ability to represent previously learned modes, so that the reconstruction error is maintained at the same level for all modes. This experiment verifies that the JMSDL method can adapt to new modes without "catastrophic forgetting."

Next, we use D_3 to monitor the CSTD process. To verify the ability of the method to monitor faults, we add faulty data to the test data. We set up three cases of faults, which occur in each of these three modes, as described in Table III. With 250 faulty samples added to the test data for each mode, there are a total of 1500 samples in the test data, containing a total of 750 normal data and 750 faulty data from three modes. The parameters of the compared method are set as follows. The CPV of the mPCA method is 0.85. For all the DL-based

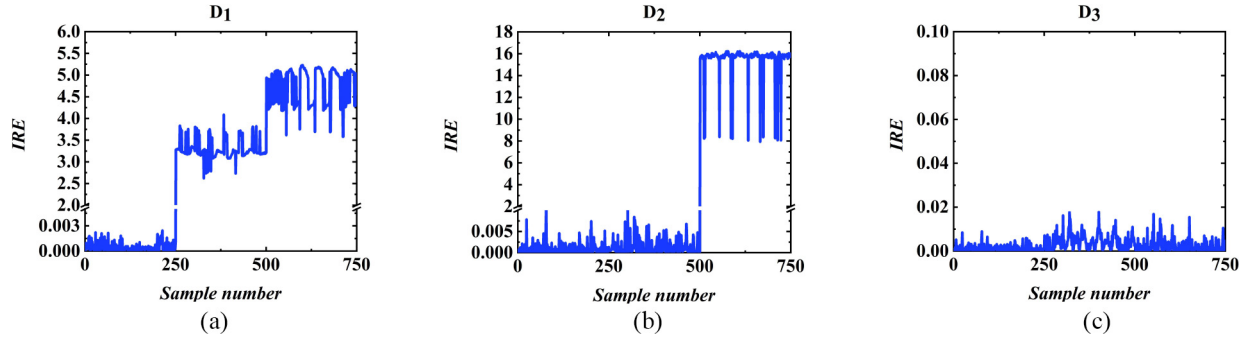


Fig. 11. Representation effect of different dictionaries on CSTH process test data. (a) IRE of D_1 to test data. (b) IRE of D_2 to test data. (c) IRE of D_3 to test data. JMSDL can continuously learn multimode data of CSTH without “catastrophic forgetting.”

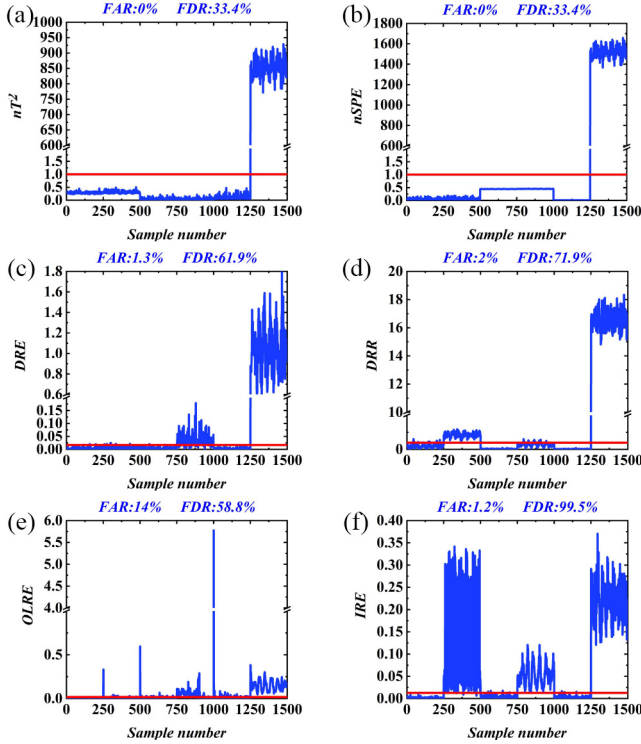


Fig. 12. Process monitoring effects of different methods on CSTH process data. (a) nT^2 statistic of the mPCA method. (b) $nSPE$ statistic of the mPCA method. (c) DRE statistic of the DL method. (d) DRR statistic of the LCDL method. (e) OLRE statistic of the ODL method. (f) IRE statistic of the proposed JMSDL method.

methods, the number of atoms is 80 and the sparsity is 4. The confidence level for all methods is 0.99.

Fig. 12 shows the process monitoring effects on the CSTH process. The mPCA method can only detect the multiplicative fault on temperature; the DL method can not detect bias fault on level; the LCDL method can not detect multiplicative fault on flow; the ODL method has large reconstruction errors for new mode data, which can be reduced only after learning some test data of this mode online. These deficiencies of these methods result in a larger FAR or smaller FDR. The proposed JMSDL method has the same representation ability as normal data of each mode, so quantitatively the JMSDL method has the best monitoring effect on the CSTH process.

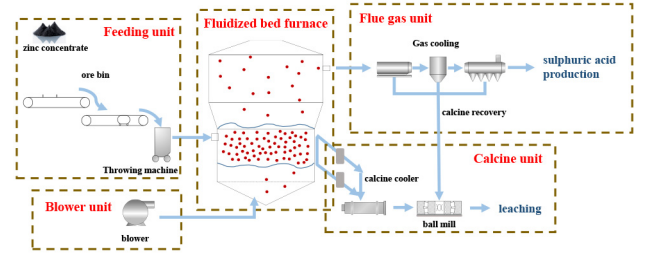


Fig. 13. Structure of the roasting process.

IV. REAL ROASTING PROCESS EXAMPLE

In order to verify the feasibility of the proposed JMSDL method in the real industrial process, we conduct an experiment on a real roasting process from a zinc smelting factory in Hunan Province, China. Roasting is the first step in the zinc smelting process. The stable and safe operation of the roasting process is highly important to reduce industrial pollution and ensure the quality of the output zinc. The main function of the roasting process is to oxidize the raw ore at a high temperature so that the insoluble zinc sulfide is converted into zinc oxide soluble in weak acids, and the output material zinc roast is transported to the subsequent leaching process. The structure of the roasting process is shown in Fig. 13. It makes sense to perform process monitoring for the roasting process to ensure a stable and safe operation. In addition, the roasting process is a typical multimode process, including efficient condition, healthy condition, over-decomposition condition, under-oxidation condition, etc, which is well suited to be modeled and monitored using the proposed method.

We collected 25-dimensional variable data of the roasting process, including temperature, flow, and pressure at several locations in the roaster. By analyzing the big data of these physical quantities, the potential features of the roasting process can be extracted. The data of four working conditions are selected as normal data. The efficient condition is mode 1, the efficient over-decomposition condition is mode 2, the efficient under-oxidation condition is mode 3, and the healthy condition is mode 4. Each mode has 1000 samples as training data and 500 samples as test data. Therefore, there are 4000 normal samples in the training set and 2000 normal samples in the test set.

D_1 is trained using the data from mode 1, where the number of atoms in the dictionary is 50. Next, we use data from mode

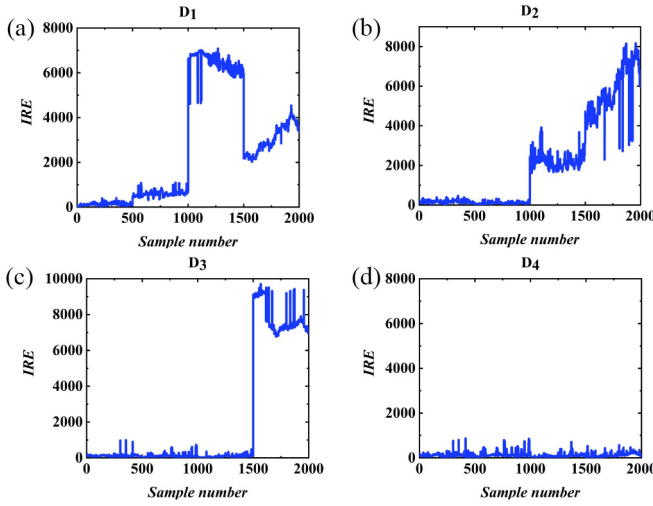


Fig. 14. Representation effect of different dictionaries on roasting process test data. (a) IRE of D_1 to test data. (b) IRE of D_2 to test data. (c) IRE of D_3 to test data. (d) IRE of D_4 to test data.

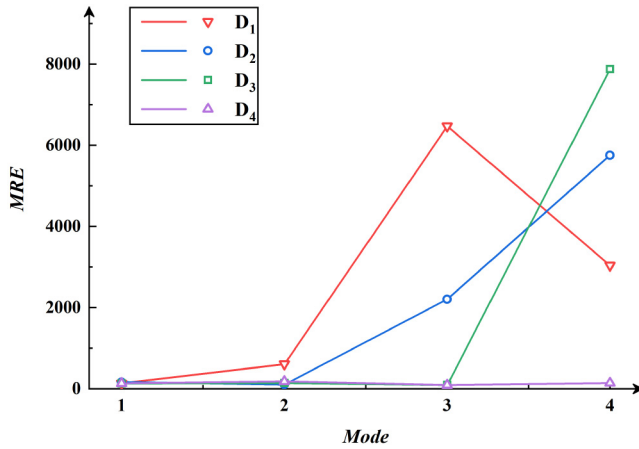


Fig. 15. Mean reconstruction errors of the four dictionaries for four modes of roasting process data. For the roasting process, JMSDL can continuously learn new modes and maintain a small MRE for the previous modes.

2, 3, and 4 to update the dictionary and obtain D_2 , D_3 , and D_4 separately. Each time the JMSDL method is used, λ_1 is 10, 10, and 6, respectively. The sparsity of all dictionaries is 3.

Fig. 14 shows the data representation effect of each dictionary on the test data. After updating, the dictionary is able to represent the data of the current mode as well as the data of the previous modes, and the reconstruction errors of all trained modes are maintained at the same level. Fig. 15 quantitatively shows the representation effect of the dictionaries of the four stages to the four modes. After learning a new mode, the MRE of the dictionary for that mode decreases rapidly, so D_4 has small reconstruction errors for all modes data. The experimental results show that the JMSDL method does not cause “catastrophic forgetting” in the roasting process, which verifies the practicality of the proposed method in real industrial processes.

Next, we use data from the roasting process to test the effectiveness of the proposed method for process monitoring. This experiment uses the efficient bed deposition condition as the

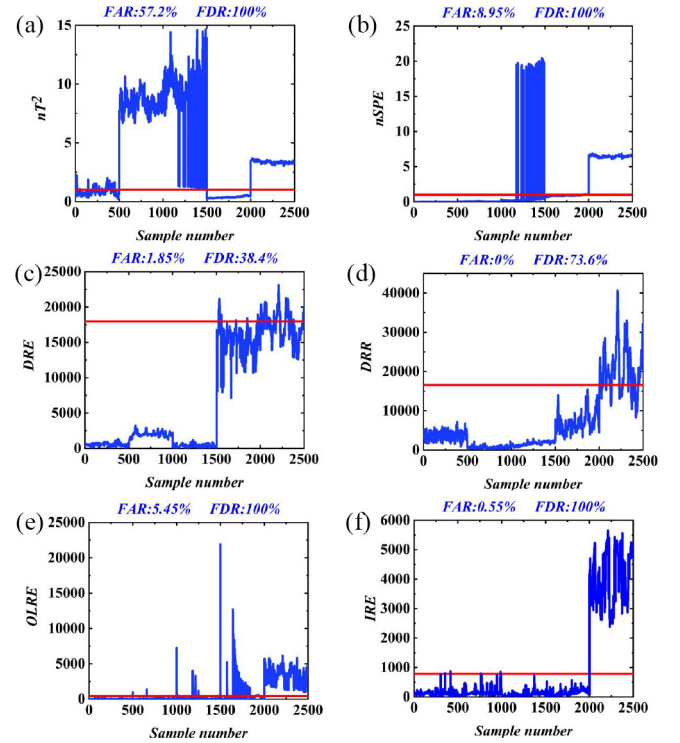


Fig. 16. Process monitoring effects of different methods on roasting process data. (a) nT^2 statistic of the mPCA method. (b) $nSPE$ statistic of the mPCA method. (c) DRE statistic of the DL method. (d) DRR statistic of the LCDL method. (e) OLRE statistic of the ODL method. (f) IRE statistic of the proposed JMSDL method.

faulty condition, and 500 faulty data are added to the end of the test data. A total of 2500 test samples. The training data remain unchanged. The control limit is first calculated based on the reconstruction errors of D_4 to the training data, and then the volume of the reconstruction error of the test data is compared with the control limit to monitor whether a fault occurs. The mPCA, DL, LCDL, and ODL methods are used as compared methods. The CPV of the mPCA method is 0.85. For all the DL-based methods, the number of atoms is 50 and the sparsity is 3. The confidence level for all methods is 0.99.

Fig. 16 shows the effectiveness of the proposed method and the compared methods for monitoring the roasting process. The mPCA method can effectively monitor faults, but cannot represent some modes well. The DL method is unable to represent mode 4, resulting in a very low FDR. The LCDL method has different levels of reconstruction errors for each mode, resulting in a higher control limit. Although the FAR is small, the FDR is also reduced. The ODL method is prone to “catastrophic forgetting” and therefore has high reconstruction errors when a new mode arrives, resulting in a high FAR. The proposed method can represent the normal data of each mode well, while distinguishing the faults, and has a good monitoring effect.

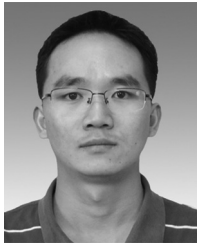
V. CONCLUSION

This article studies the problem of modeling and monitoring multimode industrial processes. In actual industrial processes,

changes in factors, such as techniques and strategies of production often lead to the creation of new modes, causing the monitoring model to often perform poorly. At the same time, updating the model with new mode data may lead to “catastrophic forgetting,” resulting in a poor representation of historical modes. To address these problems, an adaptive multimode process modeling and monitoring method based on JMSDL is developed in this article. An initial model is first built using the dictionary learning method. When a new mode is generated, the model is updated by the proposed JMSDL method so that the model continuously adapts to the new mode while maintaining the ability to represent historical data. Experiments based on a numerical simulation, the CSTDH process, and a roasting process have demonstrated that the model can adapt to the new mode without “catastrophic forgetting” and the proposed method significantly outperforms the state-of-the-art methods. It is worth mentioning the JMSDL method is a supervised method, which is one of our limitations. In addition, in the next work, we will perform a more fine-grained processing, such as considering the different extent of similarity between each atom, to further upgrade this work.

REFERENCES

- [1] J. Liu *et al.*, “Toward robust fault identification of complex industrial processes using stacked sparse-denoising autoencoder with Softmax classifier,” *IEEE Trans. Cybern.*, early access, Sep. 22, 2021, doi: [10.1109/TCYB.2021.3109618](https://doi.org/10.1109/TCYB.2021.3109618).
- [2] W. Yu, C. Zhao, and B. Huang, “MoniNet with concurrent analytics of temporal and spatial information for fault detection in industrial processes,” *IEEE Trans. Cybern.*, early access, Feb. 5, 2021, doi: [10.1109/TCYB.2021.3050398](https://doi.org/10.1109/TCYB.2021.3050398).
- [3] J. Yu and X. Yan, “Whole process monitoring based on unstable neuron output information in hidden layers of deep belief network,” *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 3998–4007, Sep. 2020.
- [4] J. Zhang, H. Chen, S. Chen, and X. Hong, “An improved mixture of probabilistic PCA for nonlinear data-driven process monitoring,” *IEEE Trans. Cybern.*, vol. 49, no. 1, pp. 198–210, Jan. 2019.
- [5] P. Goodall, D. Pantazis, and A. West, “A cyber physical system for tool condition monitoring using electrical power and a mechanistic model,” *Comput. Ind.*, vol. 118, Jun. 2020, Art. no. 103223.
- [6] D. Gorinevsky, “Fault isolation in data-driven multivariate process monitoring,” *IEEE Trans. Control Syst. Technol.*, vol. 23, no. 5, pp. 1840–1852, Sep. 2015.
- [7] J. Jiang and Q. Jiang, “Variational Bayesian probabilistic modeling framework for data-driven distributed process monitoring,” *Control Eng. Pract.*, vol. 110, May 2021, Art. no. 104778.
- [8] W. Li, H. H. Yue, S. Valle-Cervantes, and S. J. Qin, “Recursive PCA for adaptive process monitoring,” *J. Process Control*, vol. 10, no. 5, pp. 471–486, 2000.
- [9] S. Yin, G. Wang, and H. Gao, “Data-driven process monitoring based on modified orthogonal projections to latent structures,” *IEEE Trans. Control Syst. Technol.*, vol. 24, no. 4, pp. 1480–1487, Jul. 2016.
- [10] R. Guo and N. Zhang, “A process monitoring scheme for uneven-duration batch process based on sequential moving principal component analysis,” *IEEE Trans. Control Syst. Technol.*, vol. 28, no. 2, pp. 583–592, Mar. 2020.
- [11] K. Huang, S. Wu, F. Li, C. Yang, and W. Gui, “Fault diagnosis of hydraulic systems based on deep learning model with multirate data samples,” *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Jun. 10, 2021, doi: [10.1109/TNNLS.2021.3083401](https://doi.org/10.1109/TNNLS.2021.3083401).
- [12] G. Chen and Z. Ge, “SVM-tree and SVM-forest algorithms for imbalanced fault classification in industrial processes,” *IFAC J. Syst. Control*, vol. 8, Jun. 2019, Art. no. 100052.
- [13] Q. Jiang, X. Yan, and B. Huang, “Neighborhood variational Bayesian multivariate analysis for distributed process monitoring with missing data,” *IEEE Trans. Control Syst. Technol.*, vol. 27, no. 6, pp. 2330–2339, Nov. 2019.
- [14] M. Aharon, M. Elad, and A. Bruckstein, “K-SVD: An algorithm for designing overcomplete dictionaries for sparse representation,” *IEEE Trans. Signal Process.*, vol. 54, no. 11, pp. 4311–4322, Nov. 2006.
- [15] Z. Jiang, Z. Lin, and L. Davis, “Label consistent K-SVD: Learning a discriminative dictionary for recognition,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 11, pp. 2651–2664, Nov. 2013.
- [16] J. Mairal, F. Bach, J. Ponce, G. Sapiro, and A. Zisserman, “Supervised dictionary learning,” *Advances in Neural Information Processing Systems*, vol. 21. Red Hook, NY, USA: Curran Assoc., 2008.
- [17] Q. Zhang and B. Li, “Discriminative K-SVD for dictionary learning in face recognition,” in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2010, pp. 2691–2698.
- [18] K. Huang, Z. Tao, B. Sun, C. Yang, and W. Gui, “Industrial process modeling and monitoring based on jointly specific and shared dictionary learning,” *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–11, Nov. 2021.
- [19] K. Huang, L. Zhang, B. Sun, X. Liang, C. Yang, and W. Gui, “A latent feature oriented dictionary learning method for closed-loop process monitoring,” *ISA Trans.*, to be published.
- [20] X. Peng, Y. Tang, W. Du, and F. Qian, “Multimode process monitoring and fault detection: A sparse modeling and dictionary learning method,” *IEEE Trans. Ind. Electron.*, vol. 64, no. 6, pp. 4866–4875, Jun. 2017.
- [21] K. Huang, Y. Wu, H. Wen, Y. Liu, C. Yang, and W. Gui, “Distributed dictionary learning for high-dimensional process monitoring,” *Control Eng. Pract.*, vol. 98, May 2020, Art. no. 104386.
- [22] K. Huang, H. Wen, H. Ji, L. Cen, X. Chen, and C. Yang, “Nonlinear process monitoring using kernel dictionary learning with application to aluminum electrolysis process,” *Control Eng. Pract.*, vol. 89, pp. 94–102, Aug. 2019.
- [23] H. Ma, Y. Hu, and H. Shi, “A novel local neighborhood standardization strategy and its application in fault detection of multimode processes,” *Chemometr. Intell. Lab. Syst.*, vol. 118, pp. 287–300, Aug. 2012.
- [24] C. Tong, A. Palazoglu, and X. Yan, “An adaptive multimode process monitoring strategy based on mode clustering and mode unfolding,” *J. Process Control*, vol. 23, no. 10, pp. 1497–1507, 2013.
- [25] S. Zhao, J. Zhang, and Y. Xu, “Monitoring of processes with multiple operating modes through multiple principle component analysis models,” *Ind. Eng. Chem. Res.*, vol. 43, no. 22, pp. 7025–7035, 2004.
- [26] S. Natarajan and R. Srinivasan, “Multi-model based process condition monitoring of offshore oil and gas production process,” *Chem. Eng. Res. Design*, vol. 88, nos. 5–6, pp. 572–591, 2010.
- [27] C. Zhao, F. Wang, Z. Mao, N. Lu, and M. Jia, “Improved batch process monitoring and quality prediction based on multiphase statistical analysis,” *Ind. Eng. Chem. Res.*, vol. 47, no. 3, pp. 835–849, 2008.
- [28] D. Hwang and C. Han, “Real-time monitoring for a process with multiple operating modes,” *Control Eng. Pract.*, vol. 7, no. 8, pp. 891–902, 1999.
- [29] X. Xu, L. Xie, and S. Wang, “Multimode process monitoring with PCA mixture model,” *Comput. Elect. Eng.*, vol. 40, no. 7, pp. 2101–2112, 2014.
- [30] C. Ning, M. Chen, and D. Zhou, “Sparse contribution plot for fault diagnosis of multimodal chemical processes,” *IFAC-PapersOnLine*, vol. 48, no. 21, pp. 619–626, 2015.
- [31] J. C. Jeng, “Adaptive process monitoring using efficient recursive PCA and moving window PCA algorithms,” *J. Taiwan Inst. Chem. Eng.*, vol. 41, no. 4, pp. 475–481, 2010.
- [32] K. Huang, Y. Wu, C. Long, H. Ji, and C. Yang, “Adaptive process monitoring via online dictionary learning and its industrial application,” *ISA Trans.*, vol. 114, pp. 399–412, Aug. 2021.
- [33] S. Sahoo and A. Makur, “Signal recovery from random measurements via extended orthogonal matching pursuit,” *IEEE Trans. Signal Process.*, vol. 63, no. 10, pp. 2572–2581, May 2015.
- [34] C. Giraud-Christophe, “A note on the utility of incremental learning,” *AI Commun.*, vol. 13, no. 4, pp. 215–223, 2000.
- [35] K. Huang, Y. Wu, C. Wang, Y. Xie, and W. Gui, “A projective and discriminative dictionary learning for high-dimensional process monitoring with industrial applications,” *IEEE Trans. Ind. Informat.*, vol. 17, no. 1, pp. 558–568, Jan. 2021.
- [36] J. Mairal, F. Bach, J. Ponce, and G. Sapiro, “Online dictionary learning for sparse coding,” in *Proc. 26th Annu. Int. Conf. Mach. Learn.*, 2009, pp. 689–696.
- [37] Z. Ge and Z. Song, “Process monitoring based on independent component analysis—Principal component analysis (ICA-PCA) and similarity factors,” *Ind. Eng. Chem. Res.*, vol. 46, no. 7, pp. 2054–2063, 2007.



Keke Huang received the B.A. degree in automatic control from Northeastern University, Shenyang, China, in 2012, and the Ph.D. degree in control science and engineering from Tsinghua University, Beijing, China, in 2017.

He was an Associate Professor with Central South University, Changsha, China, from April 2017 to September 2021. He is currently a Full Professor with the School of Automation, Central South University, Changsha. His research interests include industrial big data, process monitoring and control, and network sciences.



Chunhua Yang (Fellow, IEEE) received the M.S. degree in automatic control engineering and the Ph.D. degree in control science and engineering from Central South University, Changsha, China, in 1988 and 2002, respectively.

Since 1999, she has been a Full Professor with the School of Information Science and Engineering, Central South University. She is currently the HoD with the School of Automation. Her current research interests include modeling and optimal control of complex industrial processes, and intelligent control systems.



Zui Tao received the B.A. degree in automation from Central South University, Changsha, China, in 2021, where he is currently pursuing the M.A. degree in control science and engineering.

His research interests include dictionary learning and process monitoring.



Weihua Gui received the B.Eng. degree in electrical engineering and the M.S. degree in automatic control engineering from Central South University, Changsha, China, in 1976 and 1981, respectively.

Since 2013, he has been an Academician of the Chinese Academy of Engineering. He is currently with the School of Automation, Central South University. His current research interests include modeling and optimal control of complex industrial processes, fault diagnoses, and distributed robust control.



Yishun Liu received the B.A. degree in automatic control from Central South University, Changsha, China, in 2017, where he is currently pursuing the Ph.D. degree in control science and engineering.

His research interests include process monitoring, machine learning, and supply chain optimization.



Shiyan Hu (Senior Member, IEEE) received the Ph.D. degree in computer engineering from Texas A&M University, Kingsville, TX, USA, in 2008.

He is a Professor and the Chair of Cyber-Physical System Security with the University of Southampton, Southampton, U.K. His research interests include cyber-physical systems and cyber-physical system security, where he has published more than 150 refereed papers, including more than 60 in premier IEEE Transactions.

Prof. Hu is the Chair of IEEE Technical Committee on Cyber-Physical Systems. He is the Editor-in-Chief of *IET Cyber-Physical Systems: Theory & Applications*. He serves as an Associate Editor for IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS, IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS, *ACM Transactions on Design Automation of Electronic Systems*, and *ACM Transactions on Cyber-Physical Systems*. He has served as a Guest Editor for eight IEEE/ACM Journals, such as *Proceedings of the IEEE* and IEEE TRANSACTIONS ON COMPUTERS. He has held chair positions in many IEEE/ACM conferences. He is a Fellow of IET and British Computer Society.



Bei Sun received the Ph.D. degree in control science and engineering from Central South University, Changsha, China, in 2015.

From 2012 to 2014, he was with the Department of Electrical and Computer Engineering, Polytechnic School of Engineering, New York University, New York, NY, USA. He is currently an Associate Professor with Central South University. His research interests include data-driven modeling, optimization, and control of nonferrous metallurgical processes.